Faculty of Engineering & Technology

Electrical & Computer Engineering Department

ENCS5342 — Information Retrieval with Applications of NLP

Assignment 2: N-gram Language Models

Prepared by:

Waad Ziadeh	**1220423**	**Sec1**
Noor Battat	**1210075**	**Sec3**

Instructor: Dr. Ahmad Shawahna

Date :30/4/2026

Table of Contents

Table of Figures

List of Tables

Task 1— Let's Build a Language Model!

Step 1 — Tokenize the Text

Question 1.A

What is the effect of this tokenization step? Describe the changes made to the text. (Hint: examine the file nonbreaking_prefix.en.)?

The Moses tokenizer ('tokenizer.pl') normalizes the text so that downstream tools see consistent units. Concretely it:

1. **Separates punctuation from adjacent words:**
   Tokens such as word. , word, , (word), and "word" are transformed into word ., word ,, (word), and " word ". This ensures that punctuation marks are treated as independent tokens rather than being attached to words.
2. **Splits contractions and possessives:**
   Contractions like don't are split into do n't, and possessives like parliament's become parliament 's. this helps models better capture grammatical structure and word patterns.
3. **Protects abbreviations:**
   Abbreviations listed in nonbreaking_prefix.en (such as Mr, Dr, St, Jan, vs, etc) are preserved.
   The period remains attached to avoid incorrect sentence splitting, for example in Mr. Smith or etc..
4. **Normalizes whitespace:** collapses runs of spaces/tabs to single spaces.
5. The -no-escape flag turns off Moses's default HTML-style escaping (& → &, | → |), keeping the original characters in the output.

❖ **Net effect: each line becomes a sequence of clean, space-separated tokens where punctuation marks are first-class tokens.**

Question 1.B

Compare the raw and tokenized files in terms of the number of tokens (total word occurrences) and the number of types (unique tokens). Explain the observed differences given the tokenisation that was applied.

File Name	Tokens (Raw)	Tokens (Tokenized)	Type(Raw)	Type (Tokenized)
UNCorpus.train	2,600,422	2,933,681	36,465	17,860
UNCorpus.test	16,693	19,169	2,695	2,155
Bible.test	11,995	13,999	2,041	1,367

Table 1: step 1.1b results

- **Tokens increase** (e.g., 2.60M → 2.93M for UNCorpus.train, +12.8%) because punctuation that was previously glued to a word (peace., (2002), "yes") is now split into separate tokens. Each punctuation symbol is counted as its own token.
- **Types decrease** dramatically (36,465 → 17,860, roughly halved) because variants that differed only in attached punctuation now collapse to a single type. Before tokenization, peace, peace., peace,, peace;, peace? were five distinct types; after tokenization, they all map to the type peace, with the punctuation marks contributing only a handful of new types

Step 2 — Build the Language Model

Question 1.C

What is the total number of 1-gram, 2-gram, and 3-gram entries in the resulting LM file?

Order	1	2	3
Count	17,862	174,767	255,058

Table 2: step 2.1c results

(Note: 17,862 = 17,860 vocabulary types + the <s> and </s> sentence-boundary tokens added by ngram-count.)

Question 1.D

What is the purpose of each column in the 1-gram section of the LM file?

The 1-gram section uses three tab-separated columns in the standard ARPA ngram format:

Column 1: log10 P(word) - the base-10 log probability of the unigram.
Column 2: The word (token) itself.
Column 3: log10 bow(word) - the base-10 log of the backoff weight α(word). Used when the model has to back off from a higher-order n-gram in which word appears as the context.

Example: -2.653553 " -1.227169 means P **(")** = $10^{-2.65}$, with backoff weight bow **(")** = $10^{-1.23}$ for any unseen 2-gram whose first token is ".

Question 1.E

Why does the 3-gram section have only two columns instead of three?

The model is a **trigram** (-order 3) there are no 4-grams in it. A backoff weight on an n-gram is only needed if a *longer* n-gram might back off through it. Since nothing backs off through a trigram in this model, the third column (backoff weight) is unnecessary for the highest-order entries and is therefore omitted. Only the log probability and the trigram itself remain.

Step 3 — Evaluate Perplexity

Question 1.F

Why is the perplexity of the UN Corpus LM different when evaluated on the Bible test set compared to the UN test set?

The LM was trained on the **UN Corpus**, so it captures the lexical distribution and word-collocation patterns of formal political/diplomatic English. The UN test set comes from the same domain, so most of its trigrams were observed during training and receive high probability ⇒ low perplexity (~19.4). The Bible test set comes from a very different genre (religious/archaic English with different vocabulary, style, and word-order patterns). Even after the model silently skips OOV words, the in-vocabulary trigrams in Bible are rare or unseen in UN training data, so they get tiny, smoothed probabilities, driving perplexity up to ~1479.7.

Question 1.G

Compute the OOV rate (100 × OOVs ÷ words) for both the UN and the Bible test files.

OOV rate = 100 × OOVs ÷ words

UNCorpus.test: Words = 19,169 | OOVs = 16 | OOV % = ***0.083%***
Bible.test: Words = 13,999 | OOVs = 3,053 | OOV % = ***21.81%***

Question 1.H

Why is the OOV rate different for the Bible test set compared to the UN test set?

The vocabulary of LM consists exclusively of words seen in UNCorpus.train. The UN test set draws from the same domain (UN documents), so its word inventory overlaps almost entirely with the training vocabulary — only 16 out of 19,169 words are unseen (≈ 1 in 1,200). The Bible test set comes from a different domain whose lexicon contains many religious/archaic items absent from UN proceedings — thee, thou, hath, begat, plus thousands of biblical proper nouns (Jerusalem, Pharaoh, Bethlehem, Levites, etc.). These never appeared during training, so 3,053 of the 13,999 Bible tokens (≈ 22 %) are flagged as OOV. The OOV rate therefore reflects domain mismatch.

Step 4 — Generate Text

Question 1.I

Provide the text generated by your run of the command above. Comment on its local fluency (does each word follow naturally from the previous one?) and its global coherence (does the overall sentence convey a meaningful idea?).

The General Service (the study ways in paragraphs 13 January 2000 / 32 thereof Torture and major groups of a successor in and long-term rehabilitation processes undertaken Assembly,

Local fluency — moderately good. Adjacent word pairs and short spans look like things one might find in a UN document: General Service, 13 January 2000, long-term rehabilitation processes, Torture and major groups, paragraphs ... thereof. This is exactly what a trigram model is supposed to give: each new word is sampled conditional on the previous two, so the model produces high-probability local transitions.

Global coherence — poor. The sentence as a whole conveys no single idea: it drifts from "General Service" to a date, to "Torture", to "rehabilitation processes", to "Assembly". A trigram model has no memory beyond a 2-word context, so it cannot maintain a topic, syntactic structure, or argument across the whole utterance. The grammatical relations break down ((the, in and long-term), confirming that the model captures local distributional regularities but not sentence-level meaning.

```
waad@waad:/mnt/c/Users/DELL/Desktop/NLPassigment2/ENCS5342-Assignment02-T1252$ bash task1.sh
################################################
First - Tokenize the texts
perl tokenizer.pl -no-escape < ../English-Mix/UNCorpus.train > ../English-Mix/UNCorpus.train.tok
Tokenizer Version 1.1
Language: en
Number of threads: 1
perl tokenizer.pl -no-escape < ../English-Mix/UNCorpus.test > ../English-Mix/UNCorpus.test.tok
Tokenizer Version 1.1
Language: en
Number of threads: 1
perl tokenizer.pl -no-escape < ../English-Mix/Bible.test > ../English-Mix/Bible.test.tok
Tokenizer Version 1.1
Language: en
Number of threads: 1
################################################
Second - Create the LM
ngram-count -text English-Mix/UNCorpus.train.tok -order 3 -lm English-Mix/UNCorpus.train.tok.3.lm -addsmooth 0.01
################################################
Third - Compute Perplexity
ngram -lm English-Mix/UNCorpus.train.tok.3.lm -order 3 -ppl English-Mix/UNCorpus.test.tok
file English-Mix/UNCorpus.test.tok: 500 sentences, 19169 words, 16 OOVs
0 zeroprobs, logprob= -25308.24 ppl= 19.39789 ppl1= 20.95907
ngram -lm English-Mix/UNCorpus.train.tok.3.lm -order 3 -ppl English-Mix/Bible.test.tok
file English-Mix/Bible.test.tok: 500 sentences, 13999 words, 3053 OOVs
0 zeroprobs, logprob= -36285.73 ppl= 1479.676 ppl1= 2065.266
################################################
Fourth - For fun, generate some random UN resolution texts!
ngram -lm English-Mix/UNCorpus.train.tok.3.lm -gen 1
( b ) of Cambodia social justice and the international community to terrorism fullest December 2001 , as coordinator sys
tem in Paris so that date on Civil and Peace and officials Prevention in order to meet of the World Programme ;
```

Figure 1: Run task 1

Task 2 — How Many Ways to Build a Language Model?

Experiment 2.1 — Effect of Tokenization

Question 2.1.A

Test file	Words	OOV	OOV %	PPL
UNCorpus.test	16693	119	0.7129	9.594028
UNCorpus.test.tok	19169	16	0.0835	8.629378
UNCorpus.test.tok.lc	19169	11	0.0574	8.86229
UNCorpus.test.tok.lc.port	19447	0	0.0000	8.83844
UNCorpus.test.tok.lc.bpe	19447	0	0.0000	8.83844
Bible.test	11995	3607	30.0709	1998.74
Bible.test.tok	13999	3053	21.8087	1369.418
Bible.test.tok.lc	13999	2797	19.9800	1067.567
Bible.test.tok.lc.port	13999	2480	17.7156	1134.46
Bible.test.tok.lc.bpe	19192	159	0.8285	3660.273
Fair.test	4388	1849	42.1376	2813.184
Fair.test.tok	5648	1761	31.1792	2141.772
Fair.test.tok.lc	5648	1559	27.6027	2630.08
Fair.test.tok.lc.port	5648	1465	25.9384	2495.547
Fair.test.tok.lc.bpe	8123	257	3.1639	8216.88

Table 3:Effect of Tokenization result

Question 2.1.B

What is the relationship between tokenization choice and OOV rate for in-domain versus out-of-domain test sets?

Each successive normalization step shrinks the OOV rate, and the effect is much more pronounced on out-of-domain text:

- **Moses tokenization** strips punctuation off words, so peace, and peace are no longer different types — UN-test OOV drops 0.71 % → 0.08 %, Bible drops 30 % → 22 %, Fair drops 42 % → 31 %.
- **Lowercasing** collapses casing variants (Peace ↔ peace); on out-of-domain text this matters more (more proper nouns / sentence-initial capitals not seen in training) — Bible 22 % → 20 %, Fair 31 % → 28 %.
- **Porter stemming** further unifies inflected forms (peaces, peaceful → peac), trimming OOV again — Bible 20 % → 17.7 %, Fair 27.6 % → 25.9 %.
- **BPE** practically eliminates OOV: any unseen word is decomposed into known subword units, and in the worst case into single characters — Bible OOV crashes from 20 % to 0.83 %, Fair from 28 % to 3.2 %.

So the *qualitative* effect is the same for in-domain and out-of-domain — OOV monotonically decreases — but the *magnitude* of the gain is far larger for out-of-domain text, because that is where the vocabulary mismatch was largest.

Question 2.1.C

What is the relationship between tokenization choice and perplexity?

For **word-level tokenizations** (raw → tok → tok.lc → tok.lc.port), perplexity moves only modestly because perplexity is computed *over the tokens that remain after OOV-skipping*: a smaller surviving vocabulary leaves a denser, easier prediction problem (stemming and lowercasing each fold many surface forms into one type, which sharpens probability estimates). So in-domain UN PPL stays in the 8–10 range and Bible/Fair perplexities drift around 1000–2700.

BPE behaves very differently. It removes OOVs by splitting unknown words into subword pieces, so the model now has to predict every subword instead of skipping it. Each rare word that used to count as one OOV (skipped) is now counted as several uncertain in-vocabulary predictions, which inflates perplexity sharply on out-of-domain text (Bible 1067 → 3660, Fair 2630 → 8217). On in-domain text the BPE perplexity stays low (8.84) because almost no UN-test words need to be split.

In short: word-level tokenization choices barely change perplexity; BPE trades OOVs for many small predictions and therefore *raises* per-token perplexity on out-of-domain data even while it eliminates OOVs.

Question 2.1.D

Based on the OOV rates and perplexity values, what do the results suggest about the similarity between the Bible and the UN Corpus, versus My Fair Lady and the UN Corpus?

At every tokenization the Fair test set has a **higher** OOV rate and (with the same word-level tokenization) higher perplexity than Bible:

Table 4: task 2.1D results

	OOV % (.tok.lc)	PPL (.tok.lc)
Bible	19.98 %	1068
Fair	27.60 %	2630

Both are far from UN (in-domain UN OOV ≈ 0.06 %, PPL ≈ 9), but Fair is the more distant of the two: a stage transcript of *My Fair Lady* contains colloquial dialogue, contractions, character names and Cockney spellings that almost never appear in UN documents. Bible English shares more formal Latinate vocabulary with UN texts (shall, hereby, unto, peoples, nations), so a larger fraction of its words is observed during training. The numbers therefore say: **Bible is closer to UN than Fair is**, although both are clearly out-of-domain.

```
waad@waad:/mnt/c/Users/DELL/Desktop/NLPassigment2/ENCS5342-Assignment02-T1252$ bash task2_1.sh
>>> Training LM on UNCorpus.train  (order=3, Witten-Bell)
    UNCorpus.test  ->  16693    119     0.7129  9.594028
    Bible.test  ->  11995       3607    30.0709 1998.74
    Fair.test  ->  4388 1849    42.1376 2813.184
>>> Training LM on UNCorpus.train.tok  (order=3, Witten-Bell)
    UNCorpus.test.tok  ->  19169        16      0.0835  8.629378
    Bible.test.tok  ->  13999   3053    21.8087 1369.418
    Fair.test.tok  ->  5648     1761    31.1792 2141.772
>>> Training LM on UNCorpus.train.tok.lc  (order=3, Witten-Bell)
    UNCorpus.test.tok.lc  ->  19169     11      0.0574  8.86229
    Bible.test.tok.lc  ->  13999        2797    19.9800 1067.567
    Fair.test.tok.lc  ->  5648  1559    27.6027 2630.08
>>> Training LM on UNCorpus.train.tok.lc.port  (order=3, Witten-Bell)
    UNCorpus.test.tok.lc.port  ->  19169        7       0.0365  9.13859
    Bible.test.tok.lc.port  ->  13999   2480    17.7156 1134.46
    Fair.test.tok.lc.port  ->  5648     1465    25.9384 2495.547
>>> Training LM on UNCorpus.train.tok.lc.bpe  (order=3, Witten-Bell)
    UNCorpus.test.tok.lc.bpe  ->  19447 0       0.0000  8.83844
    Bible.test.tok.lc.bpe  ->  19192    159     0.8285  3660.273
    Fair.test.tok.lc.bpe  ->  8123      257     3.1639  8216.88

Results written to task2_1_results.tsv
Test_file                   Variant         Words  OOVs  OOV%     PPL
UNCorpus.test                               16693  119   0.7129   9.594028
Bible.test                                  11995  3607  30.0709  1998.74
Fair.test                                   4388   1849  42.1376  2813.184
UNCorpus.test.tok           .tok            19169  16    0.0835   8.629378
Bible.test.tok              .tok            13999  3053  21.8087  1369.418
Fair.test.tok               .tok            5648   1761  31.1792  2141.772
UNCorpus.test.tok.lc        .tok.lc         19169  11    0.0574   8.86229
Bible.test.tok.lc           .tok.lc         13999  2797  19.9800  1067.567
Fair.test.tok.lc            .tok.lc         5648   1559  27.6027  2630.08
UNCorpus.test.tok.lc.port   .tok.lc.port    19169  7     0.0365   9.13859
Bible.test.tok.lc.port      .tok.lc.port    13999  2480  17.7156  1134.46
Fair.test.tok.lc.port       .tok.lc.port    5648   1465  25.9384  2495.547
UNCorpus.test.tok.lc.bpe    .tok.lc.bpe     19447  0     0.0000   8.83844
Bible.test.tok.lc.bpe       .tok.lc.bpe     19192  159   0.8285   3660.273
Fair.test.tok.lc.bpe        .tok.lc.bpe     8123   257   3.1639   8216.88
```

Figure 2: Run task 2.1

Experiment 2.2 — Effect of Training Size

Question 2.2.A

Test file	Train lines	Words	OOV	OOV %	PPL
UNCorpus.test	70,000	19169	11	0.0574	8.86229
UNCorpus.test	35,000	19169	87	0.4539	17.55982
UNCorpus.test	17,500	19169	190	0.9912	21.39002
UNCorpus.test	8,750	19169	259	1.3511	30.22754
UNCorpus.test	4,375	19169	793	4.1369	84.25468
Bible.test	70,000	13999	2797	19.9800	1067.567
Bible.test	35,000	13999	3097	22.1230	794.3207
Bible.test	17,500	13999	3360	24.0017	596.2771
Bible.test	8,750	13999	3543	25.3090	455.1322
Bible.test	4,375	13999	3727	26.6233	429.3675
Fair.test	70,000	5648	1559	27.6027	2630.08
Fair.test	35,000	5648	1649	29.1962	1922.777
Fair.test	17,500	5648	1812	32.0822	1368.466
Fair.test	8,750	5648	1909	33.7996	984.7
Fair.test	4,375	5648	2087	36.9511	809.5354

Table 5:Effect of Training Size result

Question 2.2.B

What is the relationship between training size and OOV rate?

OOV rises monotonically as training shrinks for **every** test set, because a smaller training vocabulary covers fewer of the test words. The growth rate is much faster in-domain (UN-test OOV climbs from 0.06 % to 4.1 %, a 70× increase) than out-of-domain (Bible 19.98 % → 26.62 %, only 1.3×). The reason is that Bible/Fair have already exhausted the easy overlap with UN vocabulary at 70k lines; cutting the corpus further loses mostly UN-specific terms that were not in Bible/Fair anyway.

Question 2.2.C

What is the relationship between training size and perplexity for in domain data?

Strong inverse relationship. UN-test perplexity rises dramatically as training data is reduced: 8.9 → 17.6 → 21.4 → 30.2 → 84.3. With less data, fewer trigrams are seen, the model has to back off more often,

and the smoothed probabilities of the observed events become flatter. This is the classical "more data ⇒ better LM" effect on in-domain text.

Question 2.2.D

What is the relationship between training size and perplexity for out-of domain data? Is the pattern similar to or different from the in-domain case? Explain.

The pattern is **inverted**. On Bible the trigram PPL falls from 1068 → 429, and on Fair from 2630 → 810, as training shrinks. This is *not* because the model is getting better — it is an artefact of OOV skipping. SRILM ignores OOV tokens in the perplexity computation, so as the vocabulary shrinks, more of the *hard* out-of-domain words become OOV and are silently dropped, leaving only the easy common words like the, of, and to be scored. The denominator and numerator both change in a way that lowers the reported perplexity. So out-of-domain "improvement" with less training data is misleading — it reflects evaluation only on the easy residual, not a better model.

```
waad@waad:/mnt/c/Users/DELL/Desktop/NLPassigment2/ENCS5342-Assignment02-T1252$ bash task2_2.sh
>>> Training LM on first 70000 lines
    UNCorpus.test  ->  19169    11      0.0574  8.86229
    Bible.test  ->  13999       2797    19.9800 1067.567
    Fair.test  ->  5648 1559    27.6027 2630.08
>>> Training LM on first 35000 lines
    UNCorpus.test  ->  19169    87      0.4539  17.55982
    Bible.test  ->  13999       3097    22.1230 794.3207
    Fair.test  ->  5648 1649    29.1962 1922.777
>>> Training LM on first 17500 lines
    UNCorpus.test  ->  19169    190     0.9912  21.39002
    Bible.test  ->  13999       3360    24.0017 596.2771
    Fair.test  ->  5648 1812    32.0822 1368.466
>>> Training LM on first 8750 lines
    UNCorpus.test  ->  19169    259     1.3511  30.22754
    Bible.test  ->  13999       3543    25.3090 455.1322
    Fair.test  ->  5648 1909    33.7996 984.7
>>> Training LM on first 4375 lines
    UNCorpus.test  ->  19169    793     4.1369  84.25468
    Bible.test  ->  13999       3727    26.6233 429.3675
    Fair.test  ->  5648 2087    36.9511 809.5354

Results written to task2_2_results.tsv
Test_file      Train_lines  Words  OOVs  OOV%     PPL
UNCorpus.test  70000        19169  11    0.0574   8.86229
Bible.test     70000        13999  2797  19.9800  1067.567
Fair.test      70000        5648   1559  27.6027  2630.08
UNCorpus.test  35000        19169  87    0.4539   17.55982
Bible.test     35000        13999  3097  22.1230  794.3207
Fair.test      35000        5648   1649  29.1962  1922.777
UNCorpus.test  17500        19169  190   0.9912   21.39002
Bible.test     17500        13999  3360  24.0017  596.2771
Fair.test      17500        5648   1812  32.0822  1368.466
UNCorpus.test  8750         19169  259   1.3511   30.22754
Bible.test     8750         13999  3543  25.3090  455.1322
Fair.test      8750         5648   1909  33.7996  984.7
UNCorpus.test  4375         19169  793   4.1369   84.25468
Bible.test     4375         13999  3727  26.6233  429.3675
Fair.test      4375         5648   2087  36.9511  809.5354
```

Figure 3: Run task2.2

Experiment 2.3 — Effect of LM Order

Question 2.3.A

Test file	Order	Words	OOV	OOV %	PPL
UNCorpus.test	1	19169	11	0.0574	415.2004
UNCorpus.test	2	19169	11	0.0574	32.71305
UNCorpus.test	3	19169	11	0.0574	8.86229
UNCorpus.test	4	19169	11	0.0574	4.909919
UNCorpus.test	5	19169	11	0.0574	3.875133
Bible.test	1	13999	2797	19.9800	626.0066
Bible.test	2	13999	2797	19.9800	908.7421
Bible.test	3	13999	2797	19.9800	1067.567
Bible.test	4	13999	2797	19.9800	1090.773
Bible.test	5	13999	2797	19.9800	1092.756
Fair.test	1	5648	1559	27.6027	1302
Fair.test	2	5648	1559	27.6027	2365.807
Fair.test	3	5648	1559	27.6027	2630.08
Fair.test	4	5648	1559	27.6027	2646.398
Fair.test	5	5648	1559	27.6027	2647.987

Table 6:Effect of LM order result

Question 2.3.B

What is the relationship between LM order and OOV rate?

No effect. OOV rate is determined exclusively by the *vocabulary* of the LM, which is the set of unigrams seen in training. Changing the n-gram order does not add or remove any unigrams, so OOV is identical across orders 1 → 5 for every test set.

Question 2.3.C

What is the relationship between LM order and perplexity for in domain data?

Perplexity drops sharply as order increases: 415 → 32.7 → 8.9 → 4.9 → 3.9. Higher-order models capture longer dependencies, and on in-domain text those longer contexts are predictive (e.g.,

General Assembly, Security Council, Member States of the United Nations). Each additional order gives a smaller marginal gain (415 → 33 → 9 is huge, 4.9 → 3.9 is small) because most of the predictive structure is already captured by trigrams.

Question 2.3.D

What is the relationship between LM order and perplexity for out-of-domain data? Is the pattern similar to or different from the in-domain case? Explain.

Opposite pattern. On both out-of-domain sets, perplexity *rises* with order before plateauing: Bible 626 → 909 → 1068 → 1091 → 1093, Fair 1302 → 2366 → 2630 → 2646 → 2648.

The model has memorised UN-style trigrams that almost never occur in Bible or Fair. At order 1, the model is just predicting unigram frequencies, and since common function words dominate every English text, the unigram model "fits" Bible/Fair reasonably well. As order increases, the model starts to *expect* UN-specific bigrams and trigrams; when the test text doesn't supply them, predicted probabilities collapse and perplexity climbs. After order 3 the curve flattens because backoff already routes almost everything to bigram or unigram estimates.

So higher order helps in-domain dramatically but hurts (or saturates) out-of-domain — the classic over-fitting / domain-mismatch trade-off.

```
waad@waad:/mnt/c/Users/DELL/Desktop/NLPassigment2/ENCS5342-Assignment02-T1252$ bash task2_3.sh
>>> Training LM order=1
    UNCorpus.test  ->  19169    11      0.0574  415.2004
    Bible.test  ->  13999       2797    19.9800 626.0066
    Fair.test  ->  5648 1559    27.6027 1302
>>> Training LM order=2
    UNCorpus.test  ->  19169    11      0.0574  32.71305
    Bible.test  ->  13999       2797    19.9800 908.7421
    Fair.test  ->  5648 1559    27.6027 2365.807
>>> Training LM order=3
    UNCorpus.test  ->  19169    11      0.0574  8.86229
    Bible.test  ->  13999       2797    19.9800 1067.567
    Fair.test  ->  5648 1559    27.6027 2630.08
>>> Training LM order=4
    UNCorpus.test  ->  19169    11      0.0574  4.909919
    Bible.test  ->  13999       2797    19.9800 1090.773
    Fair.test  ->  5648 1559    27.6027 2646.398
>>> Training LM order=5
    UNCorpus.test  ->  19169    11      0.0574  3.875133
    Bible.test  ->  13999       2797    19.9800 1092.756
    Fair.test  ->  5648 1559    27.6027 2647.987

Results written to task2_3_results.tsv
Test_file       Order  Words  OOVs  OOV%     PPL
UNCorpus.test   1      19169  11    0.0574   415.2004
Bible.test      1      13999  2797  19.9800  626.0066
Fair.test       1      5648   1559  27.6027  1302
UNCorpus.test   2      19169  11    0.0574   32.71305
Bible.test      2      13999  2797  19.9800  908.7421
Fair.test       2      5648   1559  27.6027  2365.807
UNCorpus.test   3      19169  11    0.0574   8.86229
Bible.test      3      13999  2797  19.9800  1067.567
Fair.test       3      5648   1559  27.6027  2630.08
UNCorpus.test   4      19169  11    0.0574   4.909919
Bible.test      4      13999  2797  19.9800  1090.773
Fair.test       4      5648   1559  27.6027  2646.398
UNCorpus.test   5      19169  11    0.0574   3.875133
Bible.test      5      13999  2797  19.9800  1092.756
Fair.test       5      5648   1559  27.6027  2647.987
```

Figure 4: Run task 2.3

Experiment 2.4 — Effect of Smoothing Method

Question 2.4.A

Test file	Smoothing	Words	OOV	OOV %	PPL
UNCorpus.test	Add-1	19169	11	0.0574	860.106
UNCorpus.test	Add-0.1	19169	11	0.0574	124.0276
UNCorpus.test	Good-Turing	19169	11	0.0574	3.96369
UNCorpus.test	Witten-Bell	19169	11	0.0574	3.875133
UNCorpus.test	Kneser-Ney	19169	11	0.0574	4.022078
Bible.test	Add-1	13999	2797	19.9800	1103.608
Bible.test	Add-0.1	13999	2797	19.9800	976.8714
Bible.test	Good-Turing	13999	2797	19.9800	1491.27
Bible.test	Witten-Bell	13999	2797	19.9800	1092.756
Bible.test	Kneser-Ney	13999	2797	19.9800	561.582
Fair.test	Add-1	5648	1559	27.6027	1670.519
Fair.test	Add-0.1	5648	1559	27.6027	1819.579
Fair.test	Good-Turing	5648	1559	27.6027	3770.759
Fair.test	Witten-Bell	5648	1559	27.6027	2647.987
Fair.test	Kneser-Ney	5648	1559	27.6027	1189.075

Table 7:Effect of Smoothing Method result

Question 2.4.B

What is the relationship between the smoothing method and perplexity for in domain data?

Add-k methods are catastrophic in-domain: Add-1 gives PPL ≈ 860 and Add-0.1 still gives 124. Adding a constant probability mass to every unseen 5-gram steals far too much probability from the (very many, very confident) observed n-grams. The discount-based methods — Good-Turing, Witten-Bell, Kneser-Ney — all give in-domain PPL ≈ 4, two orders of magnitude better. Witten-Bell is marginally best (3.875), Good-Turing 3.964, Kneser-Ney 4.022; the three are essentially tied for in-domain UN text.

Question 2.4.C

What is the relationship between the smoothing method and perplexity for out-of-domain data? Is the pattern similar to or different from the in-domain case? Explain.

The picture changes completely.

- **Kneser-Ney** is by far the best on both out-of-domain sets (Bible 561.6, Fair 1189.1) — about half the perplexity of Witten-Bell. KN's lower-order distribution is built from the count of *contexts* a word appears in, not its raw frequency, so common-but-narrow UN terms (e.g. Resolution) get reasonable but not inflated lower-order probabilities, which generalises well to unseen domains.
- **Witten-Bell** is mid-pack out-of-domain (Bible 1093, Fair 2648).
- **Good-Turing** is the worst out-of-domain (Bible 1491, Fair 3771) — its high-order discounts work poorly when the test text rarely matches training contexts; SRILM also issued *discount coeff out of range* warnings, indicating count-of-counts statistics aren't reliable for such over-fitted high-order models.
- **Add-1 / Add-0.1** are not as bad out-of-domain as in-domain, because the heavy uniform smoothing they impose is closer to the true (high) entropy of unseen text.

So in-domain favours sharp discount methods (WB, GT, KN) while out-of-domain crowns Kneser-Ney as the only method that generalises well.

Question 2.4.D

Which smoothing method performs best overall? Justify your answer with reference to the results.

Kneser-Ney. It wins on both out-of-domain sets by a wide margin (Bible 562 vs ≥ 977 for every other method; Fair 1189 vs ≥ 1671 for every other method) while remaining within 4 % of the best in-domain perplexity (4.022 vs Witten-Bell's 3.875 — a difference of 0.15 in absolute PPL on UN test, which is negligible). For any deployment that has to cope with even mildly different test distributions — exactly the realistic case — Kneser-Ney is the most reliable choice. Add-k methods should not be used for n-gram language modelling at this scale; Good-Turing and Witten-Bell are competitive in-domain but degrade much faster than KN when the test domain shifts

```
waad@waad:/mnt/c/Users/DELL/Desktop/NLPassigment2/ENCS5342-Assignment02-T1252$ bash task2_4.sh
>>> Training LM with Add-1  (flags: '-addsmooth 1')
    UNCorpus.test  ->  19169    11      0.0574  860.106
    Bible.test  ->  13999       2797    19.9800 1103.608
    Fair.test  ->  5648 1559    27.6027 1670.519
>>> Training LM with Add-0.1  (flags: '-addsmooth 0.1')
    UNCorpus.test  ->  19169    11      0.0574  124.0276
    Bible.test  ->  13999       2797    19.9800 976.8714
    Fair.test  ->  5648 1559    27.6027 1819.579
>>> Training LM with Good-Turing  (flags: '')
warning: discount coeff 7 is out of range: 1.06186
warning: discount coeff 6 is out of range: 1.24362
warning: discount coeff 7 is out of range: 1.04265
    UNCorpus.test  ->  19169    11      0.0574  3.96369
    Bible.test  ->  13999       2797    19.9800 1491.27
    Fair.test  ->  5648 1559    27.6027 3770.759
>>> Training LM with Witten-Bell  (flags: '-wbdiscount')
    UNCorpus.test  ->  19169    11      0.0574  3.875133
    Bible.test  ->  13999       2797    19.9800 1092.756
    Fair.test  ->  5648 1559    27.6027 2647.987
>>> Training LM with Kneser-Ney  (flags: '-kndiscount')
    UNCorpus.test  ->  19169    11      0.0574  4.022078
    Bible.test  ->  13999       2797    19.9800 561.582
    Fair.test  ->  5648 1559    27.6027 1189.075

Results written to task2_4_results.tsv
Test_file       Smoothing     Words  OOVs  OOV%     PPL
UNCorpus.test   Add-1         19169  11    0.0574   860.106
Bible.test      Add-1         13999  2797  19.9800  1103.608
Fair.test       Add-1         5648   1559  27.6027  1670.519
UNCorpus.test   Add-0.1       19169  11    0.0574   124.0276
Bible.test      Add-0.1       13999  2797  19.9800  976.8714
Fair.test       Add-0.1       5648   1559  27.6027  1819.579
UNCorpus.test   Good-Turing   19169  11    0.0574   3.96369
Bible.test      Good-Turing   13999  2797  19.9800  1491.27
Fair.test       Good-Turing   5648   1559  27.6027  3770.759
UNCorpus.test   Witten-Bell   19169  11    0.0574   3.875133
Bible.test      Witten-Bell   13999  2797  19.9800  1092.756
Fair.test       Witten-Bell   5648   1559  27.6027  2647.987
UNCorpus.test   Kneser-Ney    19169  11    0.0574   4.022078
Bible.test      Kneser-Ney    13999  2797  19.9800  561.582
```

Figure 5:Run task 2.4

Task 3 — Guess the Language!

Question 3.1

Training

Figure 6: task 3.1 training

Predict and evaluate

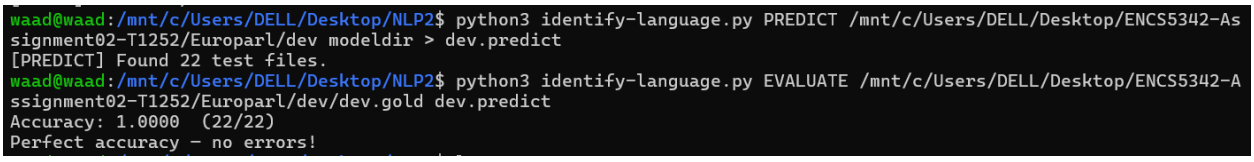

Figure 7: task 3.1 predict and ezaluate

- Results => Accuracy: 1.00 as we expected

Question 3.2

Training

Figure 8: task 3.2 training

Predict and evaluate

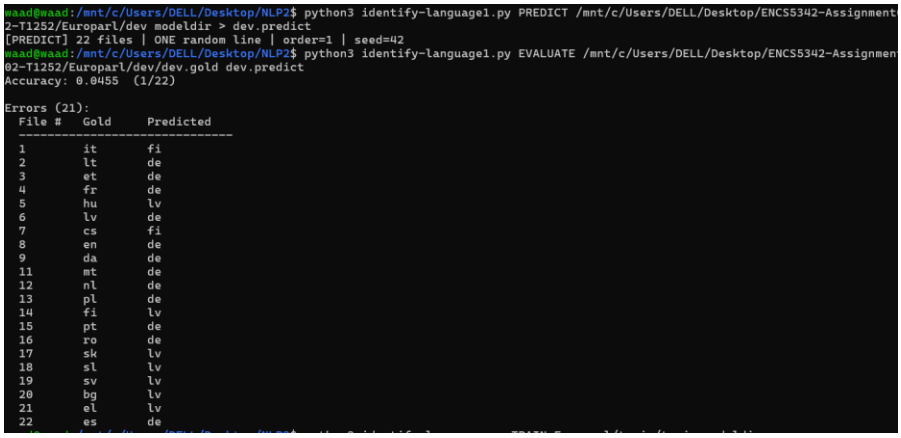

Figure 9: task 3.2 predict and training

Result => Accuracy: 0.0455

It is reasonable to expect the accuracy to go down. Is there a pattern for the errors?

The accuracy dropped significantly because the model was trained on only one random line, which is not sufficient to represent the language.

Additionally, using a unigram model ignores character sequences (history) and relies only on single-character frequencies, making it difficult to distinguish between similar languages.

We also observe that many predictions are biased toward a few languages (e.g., "de" or "lv"), indicating that the model is poorly trained.

Furthermore, errors show a clear pattern: similar languages are often confused (e.g., Slavic and Baltic languages), since their character distributions are very close without using higher-order n-grams.

Question 3.3

```
waad@waad:/mnt/c/Users/DELL/Desktop/NLP2$ python3 identify-language.py PREDICT /mnt/c/Users/DELL/Desktop/ENCS5342-Assignment02-T1252/Europarl/test modeldir > test.pred
[PREDICT] Found 15 test files.
waad@waad:/mnt/c/Users/DELL/Desktop/NLP2$ cat test.pred
test.1  sk
test.2  nl
test.3  fi
test.4  da
test.5  de
test.6  mt
test.7  ro
test.8  sl
test.9  bg
test.10 en
test.11 el
test.12 it
test.13 fr
test.14 cs
test.15 pl
waad@waad:/mnt/c/Users/DELL/Desktop/NLP2$ wc -l test.pred
15 test.pred
waad@waad:/mnt/c/Users/DELL/Desktop/NLP2$
```

Figure 10: task 3.3 results

```
#!/usr/bin/env python3
"""Identify language.py   character level language identification using SRILM.

Modes:
  TRAIN    <train-dir>/<train-base> <model-dir>   [--order N] [--lines N]
  PREDICT  <test_dir> <model dir>                 [  order N] [  lines N]
  EVALUATE <gold file> <prediction file>

The 22 TSO 639-1 labels recognised:
  bg cs da de el en es et fi fr hu it lt lv mt nl pl pt ro sk sl sv
"""

import argparse
import os
import random
import re
import subprocess
import sys
import tempfile
from pathlib import Path

LANGUAGES = [
    "bg", "cs", "da", "de", "el", "en", "es", "et", "fi", "fr", "hu",
    "it", "lt", "lv", "mt", "nl", "pl", "pt", "ro", "sk", "sl", "sv",
]

PPL_RE = re.compile(r"ppl=\s*([\d.eE+-]+)")


def char_tokenize_lines(lines):
    """Convert lines to character tokenized lines.

    Every character (including the literal space, which we replace with "_")
    becomes a separate space-delimited token, so SRILM treats characters as
    its vocabulary.
    """
    out = []
    for line in lines:
        line = line.rstrip("\n").replace(" ", "_")
        out.append(" ".join(line))
    return "\n".join(out) + "\n"


def read_lines(path, n=None, seed=None):
    with open(path, encoding="utf-8", errors="replace") as f:
        lines = [ln for ln in f.readlines() if ln.strip()]
    if n is None or n >= len(lines):
        return lines
    return random.Random(seed).sample(lines, n)


def make_tok_tmp(path, n=None, seed=None):
    lines = read_lines(path, n=n, seed=seed)
    fd, tmp_path = tempfile.mkstemp(suffix=".tok")
    os.close(fd)
    with open(tmp_path, "w", encoding="utf-8") as out:
        out.write(char_tokenize_lines(lines))
    return tmp_path


def cmd_train(args):
    Path(args.model_dir).mkdir(parents=True, exist_ok=True)
    for lang in LANGUAGES:
        train_path = f"{args.train_base}.{lang}"
        if not os.path.exists(train_path):
            print(f"  SKIP {lang} (no {train_path})", file=sys.stderr)
            continue
        seed = abs(hash(lang)) if args.lines else None
        tok_path = make_tok_tmp(train_path, n=args.lines, seed=seed)
        lm_path = os.path.join(args.model_dir, f"{lang}.lm")
        cmd = (
            f"ngram-count -text {tok_path} -order {args.order} "
            f"-wbdiscount -lm {lm_path} 2>/dev/null"
        )
        rc = os.system(cmd)
        os.unlink(tok_path)
        print(f"  {'OK  ' if rc == 0 else 'FAIL'} {lang} (order={args.order})", file=sys.stderr)


def score(tok_path, lm_path, order):
    cmd = f"ngram -lm {lm_path} -order {order} -ppl {tok_path}"
    res = subprocess.run(
        cmd, shell=True, capture_output=True, text=True
    )
    m = PPL_RE.search(res.stdout)
    return float(m.group(1)) if m else float("inf")


def cmd_predict(args):
    files = [f for f in os.listdir(args.test_dir) if re.match(r"^[^.]+\.\d+$", f)]
    files.sort(key=lambda f: int(f.rsplit(".", 1)[-1]))
    for fname in files:
        in_path = os.path.join(args.test_dir, fname)
        seed = abs(hash(fname)) if args.lines else None
        tok_path = make_tok_tmp(in_path, n=args.lines, seed=seed)
        best_lang, best_ppl = None, float("inf")
        for lang in LANGUAGES:
            lm_path = os.path.join(args.model_dir, f"{lang}.lm")
            if not os.path.exists(lm_path):
                continue
            ppl = score(tok_path, lm_path, args.order)
            if ppl < best_ppl:
                best_ppl, best_lang = ppl, lang
        os.unlink(tok_path)
        print(f"{fname}\t{best_lang}")


def _load_table(path):
    out = {}
    with open(path, encoding="utf-8") as f:
        for line in f:
            line = line.rstrip("\n")
            if not line:
                continue
            parts = line.split("\t")
            if len(parts) >= 2:
                out[parts[0]] = parts[1]
    return out


def cmd_evaluate(args):
    gold = _load_table(args.gold_file)
    pred = _load_table(args.pred_file)
    correct = 0
    total = 0
    errors = []
    for key, gold_lang in gold.items():
        if key not in pred:
            print(f"  MISSING prediction for {key}", file=sys.stderr)
            continue
        total += 1
        if pred[key] == gold_lang:
            correct += 1
        else:
            errors.append((key, gold_lang, pred[key]))
    acc = correct / total if total else 0.0
    print(f"Accuracy: {correct}/{total} = {acc:.4f}")
    if errors:
        print("Errors (file: gold  -> pred):")
        for key, g, p in errors:
            print(f"  {key}: {g} -> {p}")


def main():
    p = argparse.ArgumentParser(
        description="Character-level language identification with SRILM."
    )
    sub = p.add_subparsers(dest="mode", required=True)

    pt = sub.add_parser("TRAIN", help="Train one LM per language.")
    pt.add_argument("train_base", help="prefix; files are <train_base>.<lang>")
    pt.add_argument("model_dir")
    pt.add_argument("--order", type=int, default=3)
    pt.add_argument("--lines", type=int, default=None,
                    help="If set, sample N random training lines per language.")
    pt.set_defaults(func=cmd_train)

    pp = sub.add_parser("PREDICT", help="Predict the language of each test file.")
    pp.add_argument("test_dir")
    pp.add_argument("model_dir")
    pp.add_argument("--order", type=int, default=3)
    pp.add_argument("--lines", type=int, default=None,
                    help="If set, sample N random lines per test file before scoring.")
    pp.set_defaults(func=cmd_predict)

    pe = sub.add_parser("EVALUATE", help="Compare a prediction file to the gold.")
    pe.add_argument("gold_file")
    pe.add_argument("pred_file")
    pe.set_defaults(func=cmd_evaluate)

    args = p.parse_args()
    args.func(args)


if __name__ == "__main__":
    main()
```